

一、整合概述

OpenSSL 是一个开源的加密库，提供 TLS/SSL 协议实现及通用加密功能（对称加密、非对称加密、哈希、证书处理等）。本文档记录了 OpenSSL 在 OpenHarmony hispark_hi3519dv500_linux 产品中的适配过程。

版本信息：

- OpenSSL 版本：3.0.7
- OpenHarmony 版本：4.0-Release
- 目标产品：hispark_hi3519dv500_linux (small system)
- 目标架构：ARM64 (aarch64)
- 编译日期：2026-05-17

二、整合步骤

1. 差异分析

通过对比已适配 hi3519dv500 的参考配置与当前 SDK 仓库 `third_party/openssl`，发现 C/汇编源码完全一致，差异仅在构建配置层，涉及 3 个文件：

文件	SDK 现状	参考配置	差异类型
<code>bundle.json</code>	<code>adapted_system_type:</code> <code>["standard"]</code>	<code>["standard", "small"]</code>	必须修改
<code>BUILD.gn</code>	内联平台选择逻辑 (14-55行)	<code>import("openssl.gni")</code> ，逻辑外移	结构优化
<code>openssl.gni</code>	干净 (无 print)	有 print 调试语句	不改 (SDK 更好)

另有 `crypto/armcap.c` 存在差异（参考版本增加了 `__ARM_ARCH < 8` 守卫防止 ARMv7 平台触发未定义指令），但该改动对 hi3519dv500 (AArch64) 无功能影响，按"不影响就不改"原则不做修改。

2. 构建配置修改

改动 1: `third_party/openssl/bundle.json`

添加 `"small"` 系统类型，使 hi3519dv500 (type=small) 产品能识别该组件。

修改前：

```
"adapted_system_type": [  
  "standard"  
],
```

修改后：

```
"adapted_system_type": [  
    "standard",  
    "small"  
],
```

修改原因： hi3519dv500 产品类型为 "small"，SDK 原版仅声明 "standard"，导致构建系统跳过该组件。

改动 2: third_party/openssl/BUILD.gn

将内联的平台选择逻辑（第 16-55 行）重构为 `import("openssl.gni")`，结构更清晰。

修改前：

```
import("//build/ohos.gni")  
  
print("openssl detecting os now...")  
print("current_cpu = ${current_cpu}")  
print("current_os = ${current_os}")  
print("host_os = ${host_os}")  
openssl_selected_platform = ""  
print("is_mingw = ${is_mingw}")  
if (current_cpu == "arm" && !(current_os == "linux" || host_os == "mac")) {  
    print("openssl selected linux-armv4")  
    openssl_selected_platform = "linux-armv4"  
} else if (current_cpu == "arm64" &&  
           (!(current_os == "linux" || host_os == "mac") ||  
             current_os == "ohos")) {  
    print("openssl selected linux-aarch64")  
    openssl_selected_platform = "linux-aarch64"  
} else if ...  
    ...（共 8 个平台分支）  
}  
print("openssl detecting os done. ...")
```

修改后：

```
import("//build/ohos.gni")  
import("openssl.gni")  
  
print("openssl detecting os now...")  
print("current_cpu = ${current_cpu}")  
print("current_os = ${current_os}")  
print("host_os = ${host_os}")  
print("is_mingw = ${is_mingw}")  
  
print("openssl detecting os done. openssl_selected_platform = ${openssl_selected_platform}")
```

修改原因： 平台选择逻辑已存在于 `openssl.gni` 中，无需在 `BUILD.gn` 内重复。重构后结构更清晰。功能完全等价——hi3519dv500 编译时 `current_cpu=arm64`，都会选择 `linux-aarch64` 平台。

改动 3: vendor/hisilicon/hispark_hi3519dv500_linux/config.json

在 thirdparty 子系统下添加 openssl 组件条目，使其参与产品编译。

修改前:

```
{
  "subsystem": "thirdparty",
  "components": [
    { "component": "zlib", "features":[] }
  ]
},
```

修改后:

```
{
  "subsystem": "thirdparty",
  "components": [
    { "component": "zlib", "features":[] },
    { "component": "openssl", "features":[] }
  ]
},
```

3. 未修改文件说明

文件	差异描述	不改原因
openssl.gni	参考版本有 print 调试语句，SDK 干净	SDK 版本更优，无需同步
crypto/armcap.c	参考版本增加了 __ARM_ARCH < 8 守卫	hi3519dv500 为 AArch64 (ARMv8)，走 #else 分支，与 SDK 行为一致，不影响功能

三、问题解决记录

本次适配过程顺利，未遇到编译错误。三处改动均为构建配置层面的调整，不涉及源码修改。

四、编译验证

1. 编译命令

```
# 编译（推荐）
hb build -T openssl
```

2. 编译结果

```
[OHOS INFO] [191/194] STAMP obj/third_party/openssl/ssl_source.stamp
[OHOS INFO] [192/194] SOLINK ./libcrypto_openssl.so
[OHOS INFO] [193/194] SOLINK ./libssl_openssl.so
[OHOS INFO] [194/194] LINK ./bin/openssl
[OHOS INFO] build success
[OHOS INFO] Cost time: 0:00:51
```

3. 生成文件清单

基础路径: out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux

产物	说明
libcrypto_openssl.so	OpenSSL 加密库（共享库）
libssl_openssl.so	OpenSSL SSL/TLS 库（共享库）
bin/openssl	OpenSSL 命令行工具（可执行文件）

构建配置:

- 目标平台: hispark_hi3519dv500
- 工具链: aarch64-v01c01-linux-musl-gcc
- 平台选择: linux-aarch64 (由 openssl.gni 中 current_cpu == "arm64" 分支确定)
- 编译步骤共 194 个, 全部通过, 无 -werror 错误

五、功能说明

OpenSSL 3.0.7 提供以下核心能力:

加密库 (libcrypto)

- 对称加密: AES、DES、ChaCha20 等
- 非对称加密: RSA、ECDSA、Ed25519 等
- 哈希: SHA-1/256/512、MD5、BLAKE2 等
- 证书处理: X.509 证书解析/验证/生成
- 随机数生成: CSPRNG

SSL/TLS 库 (libssl)

- TLS 1.0/1.1/1.2/1.3 协议支持
- SSL/TLS 服务端和客户端实现

命令行工具 (openssl)

- 证书管理、加密解密、哈希计算等

六、使用方法

1. 在 GN 构建中依赖 OpenSSL

```
# 依赖加密库（共享库）
deps = [ "//third_party/openssl:libcrypto_shared" ]

# 依赖 SSL 库（共享库）
deps = [ "//third_party/openssl:libssl_shared" ]

# 依赖加密库（静态库）
deps = [ "//third_party/openssl:libcrypto_static" ]

# 依赖 SSL 库（静态库）
deps = [ "//third_party/openssl:libssl_static" ]
```

2. C 代码中使用

```
#include <openssl/ssl.h>
#include <openssl/evp.h>

// 计算SHA256
EVP_MD_CTX *ctx = EVP_MD_CTX_new();
EVP_DigestInit_ex(ctx, EVP_sha256(), NULL);
EVP_DigestUpdate(ctx, data, len);
EVP_DigestFinal_ex(ctx, md, &md_len);
EVP_MD_CTX_free(ctx);
```

七、与其他组件的关系

被依赖关系

- **openssh**: SSH 服务依赖 OpenSSL 提供 TLS/SSL 加密能力
- **curl** (如有) : HTTPS 支持
- **zlib**: OpenSSL 可选依赖 zlib 提供压缩支持

依赖关系

- **zlib** (可选) : 压缩支持

构建配置

```
组件名: openssl
子系统: thirdparty
segment destPath: third_party/openssl
adapted_system_type: ["standard", "small"]
构建目标: //third_party/openssl:openssl
```

八、注意事项

1. 系统类型适配

本次适配的核心改动是在 `bundle.json` 中添加 `"small"` 系统类型。hi3519dv500 产品类型为 `"small"`，SDK 原版仅支持 `"standard"`，不添加将导致组件被构建系统跳过。

2. BUILD.gn 重构

BUILD.gn 的重构（`import("openssl.gni")` 替代内联逻辑）为结构优化，功能完全等价。`openssl.gni` 已包含完整的平台选择逻辑，无需在 BUILD.gn 中重复。hi3519dv500 编译时两者都选择 `linux-aarch64` 平台。

3. armcap.c 差异

`crypto/armcap.c` 在参考版本中增加了 `__ARM_ARCH < 8` 守卫，用于防止 ARMv7 平台探测 ARMv8 加密指令时触发未定义指令错误。由于 hi3519dv500 为 AArch64 (ARMv8-A)，此改动不影响功能，按“不影响就不改”原则未同步。

4. -Werror 敏感度

BUILD.gn 开启了 `-werror`，hi3519dv500 交叉编译工具链（`aarch64-v01c01-linux-musl-gcc`）对警告较严格。本次适配未触发任何 `-Werror` 错误。

5. ohos_lite/BUILD.gn

`third_party/openssl/ohos_lite/BUILD.gn` 是轻量级构建文件，仅编译 4 个源文件（`cryptlib.c`、`curve25519.c`、`mem_clr.c`、`sha512.c`），适用于极简场景。本次适配使用主 BUILD.gn，提供完整的 OpenSSL 功能。

九、开发板验证

1. 编译产物确认

检查编译产物：

```
OUT=out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux
echo "=== libcrypto ===" &&
file $OUT/libcrypto_openssl.so &&
echo "=== libssl ===" &&
file $OUT/libssl_openssl.so &&
echo "=== openssl bin ===" &&
file $OUT/bin/openssl
```

预期输出（均为 ARM aarch64 ELF）：

```
libcrypto_openssl.so: ELF 64-bit LSB shared object, ARM aarch64, ...  
libssl_openssl.so: ELF 64-bit LSB shared object, ARM aarch64, ...  
bin/openssl: ELF 64-bit LSB executable, ARM aarch64, ...
```

2. 推送到开发板

```
# 推送共享库和可执行文件  
adb push libcrypto_openssl.so /usr/lib/  
adb push libssl_openssl.so /usr/lib/  
adb push bin/openssl /userdata/  
  
# 执行验证  
adb shell "chmod +x /userdata/openssl && /userdata/openssl version"
```

3. 预期输出

```
OpenSSL 3.0.7 1 Nov 2022 (Library: OpenSSL 3.0.7 1 Nov 2022)
```

判定标准：`openssl version` 正常输出版本号 = 移植成功。

十、参考资料

- OpenSSL 官网: <https://www.openssl.org/>
- OpenSSL 3.0 文档: <https://www.openssl.org/docs/man3.0/>
- OpenHarmony 文档: <https://gitee.com/openharmony>
- 参考仓库: `third_party/openssl` (已适配 hi3519dv500)